A Multi-Strategy Adaptive Lucene Architecture with Random Indexing for Enhanced TREC Retrieval

A Report on Optimizing Search and Ranking on the TREC Disks 4 and 5

Liwei Huang
School of Computer Science and Statistics
Trinity College Dublin
Dublin, Ireland
huangl3@tcd.ie

Rohan James
School of Computer Science and Statistics
Trinity College Dublin
Dublin, Ireland
jamesro@tcd.ie

Pravigya Jain
School of Computer Science and Statistics
Trinity College Dublin
Dublin, Ireland
jainpr@tcd.ie

Fengchen Hu
School of Computer Science and Statistics
Trinity College Dublin
Dublin, Ireland
fehu@tcd.ie

Yuuv Jauhari
School of Computer Science and Statistics
Trinity College Dublin
Dublin, Ireland
jauhariy@tcd.ie

Abstract

Corpus retrieval is an important research direction in the field of corpora, which can help learners or applications search for relevant content and improve learning and application efficiency. Traditional retrieval methods have problems such as high development costs, complex structural design, and low data coverage [8].

The first implementation of our Lucene-based search engine constructs standard indexers and searchers to perform information retrieval on the Cranfield document collection [13]. This method was supplemented with standard analyzers and similarity measures.

A more sophisticated system is constructed in the next iteration where we present a hybrid adaptive query generation system for full-text information retrieval on the TREC Disks 4 and 5 collections. Our system implements multiple query generation strategies, including semantic expansion using Random Indexing as a lightweight alternative to neural models, domain-specific vocabulary expansion for selected topics, and an adaptive routing mechanism that selects the optimal strategy per topic. The architecture combines five distinct query generators, selectively applied to maximize retrieval effectiveness. Evaluation demonstrates a final performance of 0.4118 MAP.

Keywords

Lucene, Custom Analyzer, Information Retrieval, TREC, Pseudo-Relevance Feedback, Rocchio's Algorithm, Random Indexing, Semantic Expansion, Domain-Specific Query Expansion, Hybrid Adaptive Retrieval, Vocabulary Mismatch

1 INTRODUCTION

The search retrieval engine is constructed using Apache Lucene, which is widely recognized as the de facto platform in industry for building search applications and has been shown to be an efficient and scalable foundation for information retrieval research, particularly when tested against standard TREC collections [15].

The document collection used for indexing and retrieval experiments was the TREC Disks 4 and 5: Retrieval Test Collections Document Set, compiled by the National Institute of Standards and Technology (NIST) for the Text REtrieval Conference (TREC) series [12]. This corpus contains approximately 2GB of text from multiple sources, including the Financial Times, Federal Register, Foreign Broadcast Information Service, and the Los Angeles Times. As per research practices, the Congressional Records have been excluded.

The following sections walk through the design of our information retrieval system. It details the parsing logic, choice of analyzers and filters, indexing methodology, query generation using query expansion, and retrieval strategy.

2 DESIGN

The design section breaks down each component of the entire information retrieval pipeline.

2.1 Document Schema and Parsers

The parsing mechanism utilizes raw string operations along with regular expressions to parse the document collection and the topics differently.

In both cases, Lombok is used in order to prevent the code from getting too verbose. It works by plugging into our build process and auto-generating Java bytecode into our .class files according to some project annotations we introduce in our code. We can plug the Project Lombok Java library into a build tool to automate some code, such as getter/setter methods and logging variables [10].

Each parser has its own schema that appends its objects to an `ArrayList`. Thereafter, the streams are passed to the indexer and searcher respectively.

The keys of the topic schema are:

- `Number`: The topic ID.
- `Title`: The title of the topic
- `Description`: The actual query or search request.

- `Narrative`: Content providing greater breadth of the description

Each document is separated by the *<DOC></DOC>* tag. In addition to that, the documents have multiple other tags and only the relevant ones are chosen for the purpose of this experiment. The chosen keys for the document schema are:

- `DOCNO`: The assigned document index.
- `DATE`: The date of the published article.
- `TITLE`: The title/headline of the article.
- `AUTHOR`: The author/authors who contributed to it.
- `TEXT`: The actual content/synopsis of the article.

2.2 Analyzer

A custom analyzer is used in the search retrieval engine. This analyzer is implemented by extending Lucene's Analyzer class. Lucene defines a set of default stopwords in its package, however, this limited set fails to filter out other commonly occurring stopwords. The sequence of stemmers and filters applied is defined in the exact sequence below:

- **`StandardTokenizer`** - A grammar-based tokenizer constructed with JFlex. This class implements the Word Break rules from the Unicode Text Segmentation algorithm [6].
- **`EnglishPossessiveFilter`** - TokenFilter that removes possessives (trailing 's) from words [2].
- **`LowerCaseFilter`** - Normalizes token text to lower case [5].
- **`ASCIIFoldingFilter`** - This class converts alphabetic, numeric, and symbolic Unicode characters which are not in the first 127 ASCII characters (the "Basic Latin" Unicode block) into their ASCII equivalents, if one exists [4].
- **`StopFilter`** - Removes stop words from a token stream [7].
- **`PorterStemFilter`** - Transforms the token stream as per the Porter stemming algorithm [3].

This design ensures that text is consistently normalized and standardized, providing efficient indexing and retrieval.

2.3 Indexer

The index is a special data structure mainly used for information content retrieval. All modern search engines are based on inverted indexes and are considered the most flexible and effective index structure. Inverted index is organized by index entries, and each index entry has a related inverted list containing data related to that entry, including which files and where they have appeared, the location and frequency of their occurrence, etc [9].

The Lucene index structure is inverted, with several segments forming each index of Lucene. Each segment is composed of multiple documents, and each document corresponds to an indexed file; Users can freely define indexes, and the general index area is composed of domain names and domain values [8].

The indexer in this report makes use of the **`EnglishAnalyzer()`** and **`StandardAnalyzer()`** for the sake of analysis. The custom analyzer in section 2.2 forms the backbone of the pipeline.

We define a method `createTextField()` that sets some defined lucene options to the newly created `FieldType()`. Below are some of the methods whose value we set to 'true' to utilize their functionalities:

- `setIndexOptions`: As we perform full-text search, we make use of the default setting where we index the documents, frequencies, and positions.
- `setTokenized`: Tokenizes the contents of the relevant field using the custom analyzer.
- `setStoreTermVectors`: The constructed indexes are now stored as a vector of identifiers.
- `setStoreTermVectorPositions`: Sets the property to also store the token positions.
- `setStoreTermVectorOffsets`: This property lets us store the token character offsets.

We create a new index configuration using `IndexWriterConfig` by passing the custom analyzer as an argument. The config's mode is set to `OpenMode.CREATE` to ensure that a new index is always created on a new run, or the existing indexes are overwritten.

Finally, calling `IndexWriter.addDocument` adds a new entry to the index file with the current parsed and indexed document.

2.4 Query Generation Strategies

Query generation is a core component of our retrieval system. Instead of a single PRF-based expansion, we implement five distinct query generators, each designed to capture different aspects of relevance and semantic information. The `HybridAdaptiveGenerator` routes topics to the optimal generator based on empirical performance, ensuring adaptive, topic-specific query formulation.

2.4.1 BasicQueryGenerator. The baseline generator constructs queries using only the `Title` and `Description` fields of each topic. No pseudo-relevance feedback or expansion is applied. Field boosts are uniform, serving as a reference point for evaluating more sophisticated strategies.

2.4.2 QueryExpansionGenerator. This generator implements a traditional Rocchio-style PRF approach with the following adaptations:

- Adaptive field boosting based on query length:
  - Short queries (≤ 3 terms): Title boost = 4.0; Description = 1.8; Narrative = 0.6
  - Medium queries (4–8 terms): Title = 2.5; Description = 1.5; Narrative = 0.5
  - Long queries (> 8 terms): Title = 2.0; Description = 1.8; Narrative = 0.8
- Feedback documents: top 18 retrieved documents per query
- Expansion terms: 42 terms selected using TF-IDF weighting
- Term filtering:
  - Minimum term length: 4 characters
  - Document frequency constraints: 0.00003 to 0.27 of the collection
  - Base query terms excluded to ensure novelty
- Rocchio weights: $\alpha = 0.47$ (original query), $\beta = 0.63$ (expansion terms)
- Negative narrative filtering excludes irrelevant or contradictory terms based on explicit cues in the topic narrative.

2.4.3 SemanticQueryGenerator. This generator augments queries with multi-component semantic signals:

- Proximity queries with slop to capture multi-term concepts occurring within a short window
- Entity extraction: capitalized terms identified as potential entities
- Co-occurrence analysis: terms that co-occur frequently with query terms are given a semantic bonus (3x multiplier)
- Expansion: 30 terms selected based on combined statistical and semantic scores
- Component weights: Base query = 0.50; Proximity = 0.20; Expansion = 0.30

2.4.4 EnhancedSemanticGenerator. This generator adapts dynamically based on query type:

- Query types detected: CAUSAL, ENTITY, COMPARATIVE, LIST, GENERAL
- Adaptive feedback: selects 12–20 top documents based on query type
- Key phrase extraction using regex patterns
- Positive and negative narrative signals parsed to emphasize relevant terms and exclude irrelevant content

2.4.5 SemanticRIGenerator. This generator combines TF-IDF, Random Indexing, and topic-specific domain terms to expand queries:

- Expansion terms:
  - 30 TF-IDF terms from top documents
  - 20 Random Indexing semantic terms
  - 20–70 domain-specific terms per topic
- Applied selectively to 17 topics
- Weights: Base = 0.50; TF-IDF = 0.30; Semantic = 0.20; Domain = 0.40–0.50

2.4.6 HybridAdaptiveGenerator (Main System). The hybrid system routes topics to the generator that empirically maximizes retrieval performance:

- 17 topics → SemanticRIGenerator
- 33 topics → QueryExpansionGenerator

This adaptive routing ensures each topic benefits from the generator best suited to its characteristics, improving overall MAP while preventing degradation from less appropriate expansion strategies.

2.5 Retrieval Engine

In information retrieval, Okapi BM25 (where BM stands for *best matching*) is a ranking function used by search engines to estimate the relevance of documents to a given search query. It is based on the probabilistic retrieval framework developed in the 1970s and 1980s. BM25 is a bag-of-words retrieval function that ranks a set of documents based on the query terms appearing in each document, regardless of their proximity within the document.

Given a query Q containing keywords $q_1, \ldots, q_n$, the BM25 score of a document D is computed as:

$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (1)$$

where:

- $f(q_i, D)$ is the frequency of keyword q_i in document D,
- $|D|$ is the length of the document D in words,
- avgdl is the average document length in the collection, and
- k_1 and b control the term frequency saturation and document length normalization respectively, where the defaults are: $k_1 = 1.2$ and $b = 0.75$.

The term $\text{IDF}(q_i)$ is the inverse document frequency weight of the query term q_i. It is usually computed as:

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right) \quad (2)$$

where:

- N is the total number of documents in the collection, and
- $n(q_i)$ is the number of documents containing the term q_i.

[1]

The retrieval engine is implemented using the `DocumentSearcher` class. We initialize Lucene's `IndexSearcher` with a pre-built inverted index[11].

The performance enhancements achieved are a result of the design of our querying strategies, more specifically the generation of it. It is known that the standard BM25 parameters do not perform very well on specific datasets. However, the innovation of our hybrid adaptive system which combines multiple generators along with random indexing and vocabulary enhancements diminishes the negative effects of the generic similarity measure.

3 SCORING AND EVALUATION

TREC Eval is used to evaluate the performance of our retrieval engine, with MAP, P@5, P@20 and nDCG@20 as the evaluation metrics. These metrics capture different aspects of retrieval performance, reflecting overall precision, top-ranked document relevance, and ranking quality. The most important metric that guided our experimental conclusion was the Mean Average Precision (MAP). The mean average precision for a set of queries is the mean of the average precision scores for each query.

$$\text{MAP} = \frac{1}{Q} \sum_{q=1}^{Q} \text{AveP}(q) \quad (3)$$

where Q is the number of queries [14].

We evaluated the retrieval engine's performance using both the EnglishAnalyzer and the Custom Analyzer with basic queries and queries enhanced by the expansion mechanism. To further improve retrieval effectiveness, we implemented the described hybrid adaptive query generation strategy and demonstrated its performance against the previous approaches.

4 RESULTS

Table 1: EnglishAnalyzer Performance

Queries	MAP	P@5	P@20	nDCG@20
Basic Queries	0.2990	0.6160	0.4860	0.5377
Queries with Expansion	0.3767	0.6720	0.5460	0.5914
Hybrid Adaptive	0.4016	0.7200	0.5900	0.6459

Table 2: Custom Analyzer Performance

Queries	MAP	P@5	P@20	nDCG@20
Basic Queries	0.3054	0.6400	0.4880	0.5426
Queries with Expansion	0.3865	0.6880	0.5500	0.6059
Hybrid Adaptive	0.4118	0.7520	0.5840	0.6537

As shown in the tables, the custom analyzer consistently outperforms the English analyzer in all query types and evaluation metrics. For all three: basic, expanded and hybrid adaptive queries, the custom analyzer scores higher in MAP, P@5, P@20, and nDCG@20, indicating that it retrieves more relevant documents and achieves better ranking. Comparing different query types, the hybrid adaptive query mechanism significantly improves all metrics for both analyzers. The custom analyzer's MAP increases from 0.3054 to 0.4118, while the English analyzer's MAP rises from 0.2990 to 0.4016, demonstrating that hybrid queries helps the system find more relevant documents. P@5 and P@20 also improve, showing that top-ranked results contain a higher proportion of relevant documents. nDCG@20 increases as well, indicating that the ranking quality of retrieved documents is enhanced. Therefore, combining the custom analyzer with multi strategy based hybrid queries can achieve the best retrieval performance. It boasts a good MAP score and excellent P@5, P@20, and nDCG@20 scores.

CONCLUSION

Our study introduces a multi strategy, hybrid adaptive search retrieval system built upon the Apache Lucene framework. This system significantly enhances retrieval effectiveness on the TREC Disks 4 and 5 collection by integrating several advanced components: a carefully engineered custom analyzer, diverse query generation strategies, semantic expansion via Random Indexing, and the incorporation of topic-specific domain vocabulary.

The custom analyzer processes text through tokenization, English possessive removal, lowercasing, ASCII folding, stopword filtering with a comprehensive list, and stemming. It is consistently applied across all query generators and outperforms the default EnglishAnalyzer.

The hybrid adaptive system includes a conjunction of five query generators: BasicQueryGenerator, QueryExpansionGenerator, SemanticQueryGenerator, EnhancedSemanticGenerator, and SemanticRIGenerator. A routing mechanism selects the optimal generator per topic, ensuring maximal relevance. Semantic expansion and domain-specific vocabulary provide significant improvements, particularly for topics prone to vocabulary mismatch.

Our final architecture achieved an MAP of 0.4118. This demonstrates and validates that proper problem diagnosis, a multi tiered query design, and targeted semantic enhancements (through vocabulary expansion) contribute more to the overall retrieval effectiveness than parameter tuning alone.

ACKNOWLEDGMENTS

We would like to acknowledge Prof. Yvette Graham for her guidance and encouragement.

References

[1] 2025. *OkapiBM25.* https://en.wikipedia.org/wiki/Okapi_BM25 Accessed: 2025-12-05.

[2] Apache Software Foundation. 2014. *EnglishPossessiveFilter — Apache Lucene 4.7.2 API Documentation.* https://lucene.apache.org/core/4_7_2/analyzers-common/org/apache/lucene/analysis/en/EnglishPossessiveFilter.html Accessed: 2025-12-04.

[3] Apache Software Foundation. 2016. *PorterStemFilter — Apache Lucene 6.3.0 API Documentation.* https://lucene.apache.org/core/6_3_0/analyzers-common/org/apache/lucene/analysis/en/PorterStemFilter.html Accessed: 2025-12-04.

[4] Apache Software Foundation. 2017. *ASCIIFoldingFilter — Apache Lucene 6.6.1 API Documentation.* https://lucene.apache.org/core/6_6_1/analyzers-common/org/apache/lucene/analysis/miscellaneous/ASCIIFoldingFilter.html Accessed: 2025-12-04.

[5] Apache Software Foundation. 2017. *LowerCaseFilter — Apache Lucene 6.6.1 API Documentation.* https://lucene.apache.org/core/6_6_1/analyzers-common/org/apache/lucene/analysis/core/LowerCaseFilter.html Accessed: 2025-12-04.

[6] Apache Software Foundation. 2017. StandardTokenizer — Apache Lucene 6.6.0 API Documentation. https://lucene.apache.org/core/6_6_0/core/org/apache/lucene/analysis/standard/StandardTokenizer.html. Accessed: 2025-12-04.

[7] Apache Software Foundation. 2018. *StopFilter — Apache Lucene 8.0.0 API Documentation.* https://lucene.apache.org/core/8_0_0/analyzers-common/org/apache/lucene/analysis/core/StopFilter.html Accessed: 2025-12-04.

[8] Fang Cai. 2024. Research on Large-scale Corpus Full-text Retrieval Based on Lucene Technology. *Procedia Computer Science* 247 (2024), 943–952. doi:10.1016/j.procs.2024.10.114

[9] Gunjan Chandwani, Anil Ahlawat, and Gaurav Dubey. 2023. An approach for document retrieval using cluster-based inverted indexing. *Journal of Information Science* 49, 3 (2023), 726–739. arXiv:https://doi.org/10.1177/01655515211018401 doi:10.1177/01655515211018401

[10] Rustam Nikkel et al. 2021. Introduction to Project Lombok. https://www.baeldung.com/intro-to-project-lombok. Accessed: 2025-02-03.

[11] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[12] Ellen M Voorhees. 1996. NIST TREC Disks 4 and 5: Retrieval Test Collections Document Set.

[13] Wikipedia contributors. 2025. *Cranfield experiments.* https://en.wikipedia.org/wiki/Cranfield_experiments Accessed: 2025-12-05.

[14] Wikipedia contributors. 2025. *Evaluation measures (information retrieval).* https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval)#Average_precision Accessed: 2025-12-04.

[15] Peilin Yang, Hui Fang, and Jimmy Lin. 2018. Anserini: Reproducible ranking baselines using Lucene. *Journal of Data and Information Quality (JDIQ)* 10, 4 (2018), 1–20.